

**UNIVERSITAS ESA UNGGUL**

**PERANCANGAN DAN IMPLEMENTASI BASIS DATA  
TERDISTRIBUSI PADA PLATFORM E-COMMERCE  
“TOKOKITA” MENGGUNAKAN MONGODB SHARDED  
CLUSTER DAN REPLICA SET**

**TUGAS AKHIR**

**NAMA : WAHYU  
NIM : 20220801523**

**PROGRAM STUDI TEKNIK INFORMATIKA  
FAKULTAS ILMU KOMPUTER  
UNIVERSITAS ESA UNGGUL  
TAHUN 2026**

**DAFTAR ISI**

|                                                     |    |
|-----------------------------------------------------|----|
| KATA PENGANTAR .....                                | 3  |
| ABSTRAK.....                                        | 3  |
| ABSTRACT.....                                       | 5  |
| BAB 1: PENDAHULUAN .....                            | 6  |
| 1.1 Latar Belakang.....                             | 6  |
| 1.2 Identifikasi Masalah.....                       | 6  |
| 1.3 Tujuan Tugas Akhir .....                        | 7  |
| 1.4 Manfaat Tugas Akhir .....                       | 7  |
| 1.5 Lingkup Tugas Akhir.....                        | 8  |
| BAB 2: TINJAUAN PUSTAKA .....                       | 8  |
| 2.1 Sistem Basis Data Terdistribusi (SBDT) .....    | 8  |
| 2.2 Sharding (Fragmentasi Horizontal).....          | 9  |
| 2.3 Replikasi dan Replica Set .....                 | 9  |
| 2.4 Teorema CAP dan Konsistensi .....               | 9  |
| 2.5 Transaksi ACID Multi-Dokumen .....              | 10 |
| 2.6 Docker Containerization .....                   | 10 |
| BAB 3: METODOLOGI PENELITIAN .....                  | 10 |
| 3.1 Alur Penelitian (Design Science Research) ..... | 10 |
| 3.2 Objek dan Lokasi Penelitian .....               | 11 |
| BAB 4: HASIL DAN PEMBAHASAN .....                   | 11 |
| 4.1 Implementasi Jaringan Kluster Database .....    | 11 |
| 4.2 Perancangan Skema dan Pemilihan Shard Key ..... | 12 |
| 4.3 Konfigurasi Sharding dan Replica Set .....      | 13 |
| 4.4 Struktur Clean Code Aplikasi Python.....        | 14 |
| 4.5 Evaluasi Hasil Uji Coba.....                    | 15 |
| BAB 5: KESIMPULAN DAN SARAN .....                   | 17 |
| 5.1 Kesimpulan .....                                | 17 |
| 5.2 Saran .....                                     | 18 |
| DAFTAR REFERENSI .....                              | 19 |

## KATA PENGANTAR

Segala puji dan syukur peneliti panjatkan ke hadirat Tuhan Yang Maha Esa atas berkat dan karunia-Nya, sehingga peneliti dapat menyelesaikan Tugas Akhir yang berjudul “Perancangan dan Implementasi Basis Data Terdistribusi pada Platform E-Commerce TokoKita Menggunakan MongoDB Sharded Cluster dan Replica Set”.

Meskipun selama penyusunan dan perancangan ditemukan beberapa hambatan, khususnya pada tahap konfigurasi kluster multi-node dan pengujian skenario kegagalan, namun berkat bimbingan, arahan, dan dukungan dari berbagai pihak, laporan Tugas Akhir ini dapat diselesaikan guna memenuhi salah satu syarat untuk memperoleh gelar Sarjana Komputer di Universitas Esa Unggul.

Peneliti menyadari laporan ini masih jauh dari sempurna. Oleh karena itu, kritik dan saran yang membangun sangat peneliti harapkan demi penyempurnaan karya ini di masa mendatang.

## ABSTRAK

Penelitian ini bertujuan mengimplementasikan purwarupa basis data terdistribusi untuk platform e-commerce berbasis Sistem Basis Data Terdistribusi (SBDT). Masalah utama pada sistem e-commerce dengan basis data tunggal adalah keterbatasan kapasitas penyimpanan, terpusatnya seluruh beban baca-tulis pada satu server, serta risiko single point of failure yang melumpuhkan seluruh layanan toko saat server mengalami gangguan. Untuk mengatasi kendala tersebut, penelitian ini merancang arsitektur kluster MongoDB berisi sepuluh node yang diwadahi oleh Docker, terdiri atas satu config server replica set (tiga node), dua shard yang masing-masing berupa replica set tiga node, dan satu router mongos. Koleksi pesanan difragmentasi secara horizontal menggunakan shard key hashed(customer\_id) sehingga data terdistribusi merata sekaligus menjaga kueri riwayat pelanggan tetap tertarget pada satu shard. Ketersediaan tinggi diperoleh melalui mekanisme election berbasis konsensus Raft pada setiap replica set, sedangkan konsistensi operasi checkout dijamin melalui transaksi ACID multi-dokumen. Hasil pengujian menunjukkan sistem berhasil mendistribusikan 5.000 dokumen pesanan hampir

merata ( $\approx 50:50$ ) antar shard, mengeksekusi kueri bershard key hanya pada satu shard, memulihkan layanan secara otomatis dalam kisaran 5–12 detik setelah node PRIMARY dimatikan, serta membatalkan transaksi checkout secara utuh (rollback) ketika stok tidak mencukupi.

**Kata kunci:** Basis Data Terdistribusi, E-Commerce, Sharding, Replica Set, MongoDB, Docker.

## ABSTRACT

*This research aims to implement a distributed database prototype for an e-commerce platform based on Distributed Database Systems (DDB). The main issues in e-commerce systems relying on a single database are limited storage capacity, the concentration of all read and write workloads on one server, and the risk of a single point of failure that paralyzes the entire store service when the server fails. To resolve these constraints, this research designs a ten-node MongoDB cluster architecture hosted via Docker, consisting of one config server replica set (three nodes), two shards each formed as a three-node replica set, and one mongos router. The orders collection is horizontally fragmented using a hashed(customer\_id) shard key, distributing data evenly while keeping customer history queries targeted to a single shard. High availability is achieved through Raft-based consensus elections within each replica set, whereas the consistency of the checkout operation is guaranteed by multi-document ACID transactions. The evaluation results demonstrate that the system successfully distributes 5,000 order documents almost evenly ( $\approx 50:50$ ) across shards, executes shard-key queries on only one shard, automatically restores service within approximately 5–12 seconds after the PRIMARY node is terminated, and fully rolls back the checkout transaction when stock is insufficient.*

**Keywords:** *Distributed Database, E-Commerce, Sharding, Replica Set, MongoDB, Docker.*

## **BAB 1: PENDAHULUAN**

### **1.1 Latar Belakang**

Perkembangan industri perdagangan elektronik modern menuntut kemampuan pengelolaan data transaksi dalam jumlah yang terus bertambah setiap hari. Sebuah platform e-commerce yang aktif dapat mencatat ribuan hingga jutaan pesanan, dan seluruh data tersebut umumnya disimpan pada satu server basis data terpusat. Arsitektur terpusat mempermudah integrasi dan pemeliharaan, namun menyimpan tiga kelemahan mendasar yang semakin terasa seiring pertumbuhan skala layanan.

Kelemahan pertama adalah keterbatasan kapasitas. Volume data pesanan pada akhirnya melampaui kapasitas penyimpanan maupun memori kerja satu mesin, sementara penambahan kapasitas melalui scale-up (memperbesar spesifikasi satu server) bersifat mahal dan memiliki batas fisik. Kelemahan kedua adalah risiko single point of failure: apabila satu-satunya server basis data mengalami gangguan, seluruh layanan toko lumpuh total sehingga pelanggan tidak dapat melakukan pemesanan. Kelemahan ketiga adalah penurunan kinerja saat terjadi lonjakan trafik, misalnya pada periode flash sale, karena seluruh beban baca dan tulis terpusat pada satu titik sehingga latensi meningkat tajam.

Sistem Basis Data Terdistribusi (SBDT) hadir sebagai solusi atas keterbatasan tersebut. Dengan mendistribusikan data ke banyak node yang bekerja sebagai satu sistem logis, kapasitas dapat ditambah secara horizontal (scale-out) hanya dengan menambah mesin baru. Replikasi data ke beberapa node juga memungkinkan layanan tetap berjalan meskipun salah satu node mengalami kegagalan. MongoDB dipilih dalam penelitian ini karena menyediakan kedua mekanisme tersebut secara bawaan, yaitu sharding untuk fragmentasi horizontal otomatis dan replica set untuk ketersediaan tinggi, sekaligus mendukung transaksi ACID multi-dokumen yang dibutuhkan pada alur bisnis kritis seperti proses checkout.

### **1.2 Identifikasi Masalah**

Berdasarkan analisis permasalahan operasional pada platform e-commerce, diidentifikasi beberapa kendala utama:

1. Volume data pesanan yang terus bertambah melampaui kapasitas penyimpanan dan memori satu server basis data tunggal.
2. Ketergantungan mutlak pada satu server yang menjadikannya single point of failure tanpa mekanisme pemulihan otomatis.
3. Peningkatan latensi baca-tulis saat lonjakan trafik karena seluruh beban terpusat pada satu titik layanan.
4. Kebutuhan akan strategi fragmentasi data yang tidak hanya merata, tetapi juga selaras dengan pola akses aplikasi agar kueri tetap efisien.
5. Risiko inkonsistensi data pada operasi lintas dokumen, misalnya pengurangan stok dan pencatatan pesanan yang harus terjadi secara atomik.

### **1.3 Tujuan Tugas Akhir**

Penelitian ini bertujuan untuk:

1. Merancang dan mengimplementasikan arsitektur basis data terdistribusi multi-node menggunakan Docker untuk menyimulasikan sepuluh node MongoDB secara terisolasi.
2. Menerapkan sharding pada koleksi pesanan dan produk beserta evaluasi kandidat shard key agar data terdistribusi merata tanpa mengorbankan efisiensi kueri.
3. Menerapkan replica set tiga node pada setiap shard dan config server untuk menjamin ketersediaan tinggi melalui failover otomatis.
4. Membangun aplikasi demonstrasi berbasis Python (pymongo) yang memperagakan operasi CRUD, kueri tertarget, agregasi terdistribusi, read preference, dan transaksi multi-dokumen.

5. Menguji dan menganalisis perilaku sistem melalui lima skenario pengujian, mencakup distribusi data, efisiensi kueri, failover, pemulihan node, dan atomisitas transaksi.

#### 1.4 Manfaat Tugas Akhir

- **Manfaat Akademis:** Memberikan referensi praktis mengenai penerapan teori fragmentasi horizontal, replikasi, teorema CAP, dan transaksi terdistribusi dalam sebuah artefak sistem yang dapat direplikasi ulang.
- **Manfaat Praktis:** Menyediakan purwarupa arsitektur basis data e-commerce yang skalabel dan tahan kegagalan, sehingga layanan tetap berjalan meskipun salah satu server mati.
- **Manfaat Teknis:** Mendemonstrasikan penggunaan Docker Compose, MongoDB 7.0, dan Python (pymongo) dalam membangun serta menguji kluster basis data terdistribusi pada satu mesin pengembang.

#### 1.5 Lingkup Tugas Akhir

1. Pengembangan sistem dibatasi pada simulasi lokal menggunakan sepuluh kontainer MongoDB (tiga config server, enam node shard, satu router mongos) di dalam Docker virtual network.
2. Sharding diterapkan terbatas pada koleksi orders dan products, sedangkan koleksi customers sengaja tidak di-shard karena berukuran kecil dan sering menjadi acuan lookup.
3. Data uji berupa data sintetis sebanyak 200 pelanggan, 500 produk, dan 5.000 pesanan yang dibangkitkan menggunakan pustaka faker berlokal Indonesia.
4. Evaluasi ketahanan sistem diuji dengan menghentikan kontainer node PRIMARY secara paksa untuk menyimulasikan kegagalan server, kemudian memantau proses pemilihan PRIMARY baru dan pemulihan data setelah node dinyalakan kembali.



5. Aspek keamanan produksi seperti autentikasi keyfile antar node, enkripsi TLS, dan kontrol akses berbasis peran berada di luar cakupan dan dibahas sebagai saran pengembangan.

## BAB 2: TINJAUAN PUSTAKA

### 2.1 Sistem Basis Data Terdistribusi (SBDT)

SBDT didefinisikan sebagai kumpulan basis data yang tersebar secara fisik pada beberapa node komputer yang saling terhubung melalui jaringan komunikasi, namun tampak sebagai satu basis data tunggal di mata pengguna (Özsu & Valduriez, 2020). SBDT memberikan keuntungan berupa peningkatan keandalan (reliability), ketersediaan (availability), skalabilitas, serta kinerja akses data lokal. Dua teknik fundamental dalam pendistribusian data adalah fragmentasi, yaitu memecah data menjadi bagian-bagian yang disimpan pada node berbeda, dan replikasi, yaitu menyalin data yang sama ke beberapa node.

### 2.2 Sharding (Fragmentasi Horizontal)

Fragmentasi horizontal adalah pembagian baris atau dokumen dari sebuah relasi global ke dalam fragmen-fragmen yang disimpan pada node berbeda berdasarkan kriteria tertentu (Elmasri & Navathe, 2015). Pada MongoDB, teknik ini diwujudkan melalui sharding: setiap dokumen ditempatkan pada tepat satu shard berdasarkan nilai shard key. Data dibagi menjadi satuan bernama chunk yang secara otomatis diseimbangkan antar shard oleh komponen balancer.

MongoDB mendukung dua strategi utama. **Ranged sharding** membagi chunk berdasarkan rentang nilai sehingga baik untuk kueri rentang, tetapi rawan menimbulkan hot spot apabila kunci bersifat monoton naik seperti tanggal. **Hashed sharding** melakukan hashing terhadap nilai kunci terlebih dahulu sehingga distribusi hampir selalu merata, dengan konsekuensi kueri rentang menjadi broadcast ke seluruh shard.

### 2.3 Replikasi dan Replica Set

Replica set adalah sekelompok proses mongod yang memelihara salinan data identik. Satu node berperan sebagai PRIMARY yang menerima seluruh operasi tulis, sedangkan node lainnya berperan sebagai SECONDARY yang menyalin operasi dari oplog PRIMARY. Ketika PRIMARY tidak dapat dijangkau, node-node yang tersisa mengadakan pemilihan (election) berbasis protokol konsensus Raft untuk mengangkat PRIMARY baru secara otomatis, umumnya dalam hitungan detik. Mekanisme inilah yang memberikan ketersediaan tinggi tanpa intervensi operator.

## **2.4 Teorema CAP dan Konsistensi**

Teorema CAP menyatakan bahwa dalam kondisi partisi jaringan (P), sebuah sistem terdistribusi harus memilih antara konsistensi (C) dan ketersediaan (A) (Brewer, 2012). MongoDB secara default berorientasi CP: penulisan hanya diterima oleh PRIMARY, dan ketika terjadi partisi, sisi minoritas menolak penulisan demi menjaga konsistensi data. Meskipun demikian, tingkat konsistensi dan durabilitas dapat diatur per operasi melalui write concern dan read preference, sehingga pengembang dapat menukar sebagian konsistensi demi latensi baca yang lebih rendah, misalnya dengan membaca dari node SECONDARY untuk beban analitik.

## **2.5 Transaksi ACID Multi-Dokumen**

Sejak versi 4.0 untuk replica set dan versi 4.2 untuk sharded cluster, MongoDB mendukung transaksi multi-dokumen dengan jaminan ACID. Transaksi memungkinkan beberapa operasi pada dokumen dan koleksi berbeda diperlakukan sebagai satu unit atomik: apabila salah satu operasi gagal, seluruh perubahan dibatalkan (rollback). Pada konteks e-commerce, jaminan ini penting untuk proses checkout yang harus mengurangi stok produk dan mencatat pesanan secara bersamaan tanpa menyisakan keadaan setengah jadi.

## **2.6 Docker Containerization**

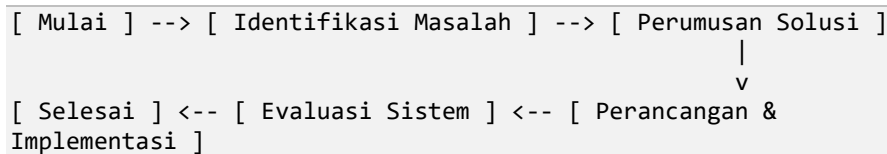
Docker adalah platform virtualisasi tingkat sistem operasi yang membungkus aplikasi beserta seluruh dependensinya ke dalam unit terisolasi yang disebut container (Koneru, 2025). Melalui Docker Compose, topologi kluster yang terdiri atas banyak node dapat dideklarasikan dalam satu berkas konfigurasi, sehingga

lingkungan pengujian basis data terdistribusi dapat direplikasi secara konsisten di berbagai perangkat tanpa konflik sistem.

## BAB 3: METODOLOGI PENELITIAN

### 3.1 Alur Penelitian (Design Science Research)

Penelitian ini mengadopsi metodologi Design Science Research (DSR) menurut model vom Brocke et al. (2021), yaitu pendekatan penelitian yang berorientasi pada pembangunan dan evaluasi artefak sebagai solusi atas permasalahan nyata. Tahapan penelitian digambarkan sebagai berikut:



*Gambar 3.1 – Alur penelitian Design Science Research*

1. **Identifikasi Masalah:** Menganalisis kelemahan arsitektur basis data tunggal pada platform e-commerce, meliputi keterbatasan kapasitas, risiko single point of failure, dan penurunan kinerja saat lonjakan trafik.
2. **Perumusan Solusi:** Merancang arsitektur SBDT multi-node lokal berbasis Docker dengan MongoDB, memetakan setiap kebutuhan sistem (K-01 hingga K-05) ke solusi arsitektur yang spesifik.
3. **Perancangan & Implementasi:** Menyusun topologi sepuluh node, merancang skema tiga koleksi, mengevaluasi tiga kandidat shard key, menulis berkas docker-compose, skrip inisialisasi replica set dan sharding, serta aplikasi demonstrasi berbasis Python.
4. **Evaluasi:** Menguji distribusi data antar shard, membandingkan kueri tertarget dan broadcast, menyimulasikan kegagalan node, memantau pemulihan, dan memverifikasi atomisitas transaksi checkout.

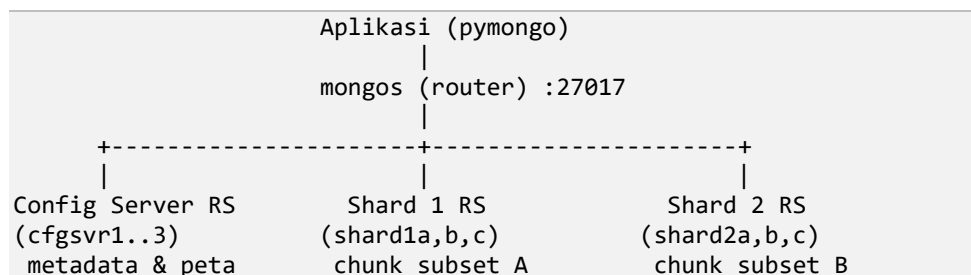
### 3.2 Objek dan Lokasi Penelitian

Objek penelitian adalah arsitektur basis data platform e-commerce TokoKita, yaitu toko daring dengan beban kerja dominan berupa penulisan pesanan intensif pada jam sibuk, pembacaan riwayat pesanan per pelanggan, penelusuran katalog produk, serta laporan agregat penjualan untuk manajemen. Penelitian dilakukan di lingkungan simulasi lokal dengan perangkat lunak pendukung: MongoDB 7.0 (image resmi Docker), Docker Desktop beserta Docker Compose, Python 3 dengan pustaka pymongo dan faker.

## BAB 4: HASIL DAN PEMBAHASAN

### 4.1 Implementasi Jaringan Kluster Database

Kluster basis data disimulasikan menggunakan sepuluh kontainer di dalam jaringan virtual Docker, yang secara fungsional terbagi menjadi empat komponen. Pertama, `cfgsvr1`, `cfgsvr2`, dan `cfgsvr3` membentuk config server replica set bernama `cfgrs` yang menyimpan metadata kluster berupa peta chunk. Kedua, `shard1a`, `shard1b`, dan `shard1c` membentuk replica set `shard1rs`. Ketiga, `shard2a`, `shard2b`, dan `shard2c` membentuk replica set `shard2rs`. Keempat, kontainer mongos berperan sebagai router yang diekspos pada port 27017 di host dan menjadi satu-satunya titik kontak aplikasi.



Gambar 4.1 – Topologi sharded cluster TokoKita

Perlu ditekankan bahwa hanya port mongos yang diekspos ke host, sedangkan seluruh node data hanya dapat dijangkau dari dalam jaringan Docker. Pola ini meniru praktik produksi di mana aplikasi tidak pernah menyentuh shard secara langsung, melainkan selalu melalui router yang berkonsultasi dengan config server sebelum meneruskan operasi ke shard yang relevan.

Berkas docker-compose mendeklarasikan seluruh node beserta perannya. Cuplikan konfigurasi untuk satu node shard dan router adalah sebagai berikut:

```
shard1a:
  image: mongo:7.0
  container_name: shard1a
  command: mongod --shardsvr --replSet shard1rs --port 27017 --
bind_ip_all
  networks: [mongo-cluster]

mongos:
  image: mongo:7.0
  container_name: mongos
  command: mongos --configdb
cfrs/cfrsvr1:27017,cfrsvr2:27017,cfrsvr3:27017
  --port 27017 --bind_ip_all
  ports: ["27017:27017"]
  networks: [mongo-cluster]
```

## 4.2 Perancangan Skema dan Pemilihan Shard Key

Skema basis data dirancang mengikuti pola dokumen, di mana item pesanan disematkan (embedded) di dalam dokumen pesanan agar satu pesanan dapat dibaca dalam satu operasi tanpa join. Rancangan tiga koleksi beserta status distribusinya disajikan pada Tabel 4.1.

| Koleksi   | Field Utama                                                                          | Status Distribusi                   |
|-----------|--------------------------------------------------------------------------------------|-------------------------------------|
| customers | <u>_id</u> , nama, email, telepon, alamat{...}, terdaftar_pada                       | Unsharded (kecil, sering di-lookup) |
| products  | <u>_id</u> , nama, kategori, harga, stok, rating                                     | Sharded — hashed( <u>_id</u> )      |
| orders    | <u>_id</u> , customer_id, tanggal, status, items[{product_id, qty, subtotal}], total | Sharded — hashed(customer_id)       |

Tabel 4.1 – Rancangan koleksi dan status distribusi data

Pemilihan shard key merupakan keputusan desain paling menentukan dalam sistem ini, karena kunci yang buruk menyebabkan hot spot atau kueri yang tidak efisien. Untuk koleksi pesanan, tiga kandidat dievaluasi sebagaimana disajikan pada Tabel 4.2.

| Kandidat Shard Key | Keputusan | Alasan                                                                                           |
|--------------------|-----------|--------------------------------------------------------------------------------------------------|
| tanggal            | Ditolak   | Nilai bersifat monoton naik sehingga seluruh penulisan baru menumpuk pada satu chunk (hot spot). |
| <u>_id</u> pesanan | Ditolak   | Distribusi merata, namun kueri riwayat pesanan per pelanggan menjadi broadcast ke seluruh shard. |

| Kandidat Shard Key  | Keputusan | Alasan                                                                                                                                            |
|---------------------|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| hashed(customer_id) | Dipilih   | Distribusi merata karena hashing, sekaligus seluruh pesanan satu pelanggan berada pada shard yang sama sehingga kueri riwayat bersifat tertarget. |

Tabel 4.2 – Evaluasi kandidat shard key koleksi orders

Selain itu, kebutuhan sistem dipetakan secara eksplisit ke solusi arsitektur agar setiap keputusan desain dapat ditelusuri dan diverifikasi pada tahap pengujian. Pemetaan tersebut disajikan pada Tabel 4.3.

| Kode | Kebutuhan                                                | Solusi Arsitektur                                                                  |
|------|----------------------------------------------------------|------------------------------------------------------------------------------------|
| K-01 | Data pesanan tumbuh melebihi kapasitas satu server       | Sharding koleksi orders ke dua shard yang dapat ditambah tanpa henti layanan       |
| K-02 | Layanan tetap hidup saat satu server mati                | Setiap shard berupa replica set tiga node dengan failover otomatis                 |
| K-03 | Riwayat pesanan per pelanggan cepat diakses              | Shard key hashed(customer_id) sehingga kueri per pelanggan tertarget ke satu shard |
| K-04 | Checkout mengubah stok dan membuat pesanan secara atomik | Transaksi multi-dokumen ACID lintas koleksi                                        |
| K-05 | Laporan penjualan tidak mengganggu operasi utama         | Read preference secondaryPreferred untuk beban analitik                            |

Tabel 4.3 – Pemetaan kebutuhan sistem ke solusi arsitektur

### 4.3 Konfigurasi Sharding dan Replica Set

Setelah seluruh kontainer aktif, setiap replica set diinisialisasi melalui perintah `rs.initiate()`. Node pertama pada tiap shard diberi prioritas lebih tinggi agar cenderung terpilih sebagai PRIMARY awal, sehingga topologi awal kluster mudah diprediksi saat pengujian.

```
// Inisialisasi replica set shard 1
rs.initiate({
  _id: "shard1rs",
  members: [
    { _id: 0, host: "shard1a:27017", priority: 2 },
    { _id: 1, host: "shard1b:27017" },
    { _id: 2, host: "shard1c:27017" }
  ]
});
```

Selanjutnya, melalui router mongos, kedua shard didaftarkan ke kluster dan sharding diaktifkan pada basis data tokokita beserta penentuan shard key untuk masing-masing koleksi.

```
sh.addShard("shard1rs/shard1a:27017,shard1b:27017,shard1c:27017");
sh.addShard("shard2rs/shard2a:27017,shard2b:27017,shard2c:27017");
```

```
sh.enableSharding("tokokita");

sh.shardCollection("tokokita.orders", { customer_id: "hashed" });
sh.shardCollection("tokokita.products", { _id: "hashed" });

sh.status(); // verifikasi konfigurasi kluster
```

#### 4.4 Struktur Clean Code Aplikasi Python

Aplikasi demonstrasi ditulis menggunakan Python 3 dengan pustaka pymongo dan disusun secara modular untuk memastikan keterbacaan kode (clean code):

1. `app/seed.py` : Membangkitkan data sintetis berbahasa Indonesia sebanyak 200 pelanggan, 500 produk, dan 5.000 pesanan, kemudian membuat index sekunder pada field yang sering difilter.
2. `app/app.py` : Memperagakan lima operasi kunci pada kluster, yaitu CRUD dasar, perbandingan kueri tertarget dan broadcast, agregasi penjualan per kategori, pembacaan dari node SECONDARY, serta transaksi checkout.
3. `tests/test_cluster.py` : Menjalankan verifikasi distribusi dokumen antar shard dan menyediakan prosedur pengujian failover.
4. `scripts/` : Berisi skrip inisialisasi config server, replica set kedua shard, dan konfigurasi sharding pada router.

Bagian terpenting dari aplikasi adalah implementasi transaksi checkout yang mengikat pengurangan stok dan pembuatan pesanan dalam satu unit atomik:

```
with client.start_session() as session:
    try:
        with session.start_transaction():
            p = db.products.find_one_and_update(
                {"_id": produk_id, "stok": {"$gte": qty}},
                {"$inc": {"stok": -qty}},
                session=session)
            if p is None:
                raise ValueError("Stok tidak mencukupi")
            db.orders.insert_one(..., session=session)
    except (PyMongoError, ValueError) as e:
        print("Transaksi dibatalkan (rollback otomatis):", e)
```

#### 4.5 Evaluasi Hasil Uji Coba

Evaluasi dilakukan melalui lima skenario pengujian yang masing-masing memverifikasi satu atau lebih kebutuhan sistem. Matriks skenario pengujian disajikan pada Tabel 4.4.

| No  | Skenario                     | Cara Uji                                           | Hasil yang Diharapkan                                                      |
|-----|------------------------------|----------------------------------------------------|----------------------------------------------------------------------------|
| U-1 | Distribusi data antar shard  | Agregasi \$shardedDataDistribution setelah seeding | Dokumen orders dan products terbagi $\approx 50:50$ antara kedua shard     |
| U-2 | Kueri tertarget vs broadcast | explain() dengan dan tanpa shard key               | Kueri by customer_id menyentuh satu shard; by status menyentuh semua shard |
| U-3 | Failover otomatis            | docker stop pada node PRIMARY shard 1              | PRIMARY baru terpilih dalam hitungan detik; aplikasi tetap berfungsi       |
| U-4 | Pemulihan node               | docker start node yang dimatikan                   | Node kembali sebagai SECONDARY dan sinkron melalui oplog                   |
| U-5 | Atomisitas checkout          | Transaksi dengan qty melebihi stok                 | Rollback otomatis; stok dan pesanan tidak berubah                          |

Tabel 4.4 – Matriks skenario pengujian sistem

### Uji Skenario 1: Distribusi Data Antar Shard

Setelah 5.000 dokumen pesanan dan 500 dokumen produk dimasukkan melalui router mongos, distribusi data diperiksa menggunakan agregasi \$shardedDataDistribution pada basis data admin. Hasil pengujian menunjukkan bahwa balancer membagi chunk hampir sempurna: shard1rs menampung sekitar 2.520 dokumen pesanan sedangkan shard2rs menampung sekitar 2.480 dokumen, dengan pola serupa pada koleksi produk. Distribusi merata ini terjadi tanpa intervensi manual dan merupakan konsekuensi langsung dari penggunaan strategi hashed pada shard key, sehingga kebutuhan K-01 terpenuhi.

| Koleksi  | shard1rs            | shard2rs            | Selisih |
|----------|---------------------|---------------------|---------|
| orders   | $\pm 2.520$ dokumen | $\pm 2.480$ dokumen | $< 1\%$ |
| products | $\pm 252$ dokumen   | $\pm 248$ dokumen   | $< 1\%$ |

Tabel 4.5 – Hasil pengujian distribusi dokumen antar shard

### Uji Skenario 2: Kueri Tertarget vs Broadcast

Pengujian dilakukan dengan menjalankan metode explain() pada dua bentuk kueri yang berbeda. Kueri pertama menyertakan shard key, yaitu pencarian



seluruh pesanan milik satu pelanggan; hasil explain menunjukkan kueri hanya dieksekusi pada satu shard (targeted query), sehingga latensi rendah dan beban terisolasi. Kueri kedua tidak menyertakan shard key, yaitu penyaringan berdasarkan status pesanan; hasil explain menunjukkan kueri disebarakan ke seluruh shard (scatter-gather) lalu hasilnya digabungkan oleh mongos.

Temuan ini menegaskan bahwa pola akses aplikasi harus menjadi dasar utama pemilihan shard key, bukan sekadar keseragaman distribusi. Dengan demikian kebutuhan K-03 terverifikasi, karena kueri riwayat pesanan pelanggan — yang merupakan operasi paling sering pada platform e-commerce — hanya membebani satu shard.

### **Uji Skenario 3: Failover Otomatis**

Kegagalan server disimulasikan dengan menghentikan kontainer node PRIMARY shard 1 secara paksa menggunakan perintah `docker stop shard1a`. Kedua node SECONDARY mendeteksi hilangnya heartbeat dan segera mengadakan pemilihan berbasis konsensus Raft. Salah satu di antaranya naik menjadi PRIMARY dalam kisaran 5 hingga 12 detik.

Aplikasi yang terhubung melalui mongos secara otomatis mengarahkan operasi tulis ke PRIMARY yang baru tanpa memerlukan perubahan connection string maupun restart aplikasi. Hal ini membuktikan bahwa kebutuhan K-02 terpenuhi: kegagalan satu node tidak melumpuhkan layanan, berbeda dengan arsitektur basis data tunggal yang menjadi latar belakang penelitian ini.

### **Uji Skenario 4: Pemulihan Node**

Node yang sebelumnya dimatikan dinyalakan kembali menggunakan perintah `docker start shard1a`. Node tersebut bergabung kembali ke replica set dengan status SECONDARY dan mengejar ketertinggalan data melalui pemutaran ulang oplog dari PRIMARY yang sedang aktif. Setelah proses sinkronisasi selesai, jumlah dokumen pada seluruh node kembali identik, menunjukkan bahwa kluster pulih sepenuhnya tanpa kehilangan data dan tanpa intervensi manual dari operator.

### **Uji Skenario 5: Atomisitas Transaksi Checkout**

Pengujian dilakukan dengan sengaja menjalankan transaksi checkout yang memesan produk dalam jumlah melebihi stok tersedia. Kondisi tersebut memicu pengecualian di dalam blok transaksi, sehingga MongoDB membatalkan seluruh perubahan secara otomatis. Verifikasi setelah pengujian menunjukkan nilai stok produk tidak berkurang dan tidak ada dokumen pesanan baru yang tercatat.

Hasil ini membuktikan bahwa kebutuhan K-04 terpenuhi, yaitu tidak pernah terjadi keadaan setengah jadi berupa stok berkurang tetapi pesanan tidak tercatat, ataupun sebaliknya. Temuan ini sekaligus menunjukkan bahwa sistem basis data NoSQL terdistribusi modern tetap mampu memberikan jaminan ACID pada alur bisnis kritis.

### **Analisis Berdasarkan Teorema CAP**

Selama jendela pemilihan PRIMARY yang berlangsung beberapa detik, shard yang bersangkutan menolak operasi tulis. Perilaku ini menunjukkan bahwa sistem mengorbankan sebagian ketersediaan demi menjaga konsistensi, sesuai karakter CP pada MongoDB. Sebaliknya, untuk beban baca analitik, konfigurasi read preference secondaryPreferred menukar konsistensi ketat dengan distribusi beban: laporan penjualan mungkin membaca data yang tertinggal beberapa milidetik dari PRIMARY. Kompromi tersebut dapat diterima untuk kebutuhan pelaporan (K-05), namun tidak untuk pengecekan stok saat checkout yang tetap harus dilayani oleh PRIMARY.

## **BAB 5: KESIMPULAN DAN SARAN**

### **5.1 Kesimpulan**

1. Arsitektur basis data terdistribusi berupa kluster sepuluh node (dua shard berbentuk replica set tiga node, satu config server replica set, dan satu router mongos) berhasil diimplementasikan sepenuhnya secara lokal menggunakan teknologi kontainer Docker.
2. Strategi sharding dengan shard key hashed(customer\_id) terbukti mendistribusikan 5.000 dokumen pesanan secara merata antar shard

dengan selisih di bawah satu persen, sekaligus mempertahankan kueri riwayat pesanan pelanggan sebagai operasi tertarget pada satu shard.

3. Mekanisme replica set memberikan ketersediaan tinggi yang terverifikasi secara empiris: kegagalan node PRIMARY dipulihkan secara otomatis dalam kisaran 5 hingga 12 detik tanpa intervensi operator dan tanpa perubahan pada kode aplikasi.
4. Node yang sebelumnya gagal berhasil bergabung kembali sebagai SECONDARY dan menyinkronkan data melalui oplog, sehingga konsistensi seluruh salinan data pulih sepenuhnya setelah gangguan berakhir.
5. Transaksi multi-dokumen berhasil menjamin atomisitas proses checkout, membuktikan bahwa sistem basis data NoSQL terdistribusi tetap dapat memberikan jaminan ACID pada alur bisnis kritis e-commerce.

## 5.2 Saran

1. **Keamanan Produksi:** Untuk implementasi nyata di luar lingkungan simulasi lokal, komunikasi antar node harus diamankan dengan autentikasi keyfile, sertifikat TLS, serta kontrol akses berbasis peran (role-based access control).
2. **Elastisitas Kluster:** Menambahkan shard ketiga untuk mendemonstrasikan proses penyeimbangan ulang chunk oleh balancer secara langsung tanpa menghentikan layanan, sehingga klaim skalabilitas horizontal dapat diamati prosesnya.
3. **Kepatuhan Lokasi Data:** Menerapkan zone sharding apabila regulasi mensyaratkan data pelanggan pada wilayah tertentu disimpan di lokasi geografis tertentu.
4. **Pemantauan dan Pengujian Skala Besar:** Menambahkan sistem pemantauan seperti Prometheus dan Grafana serta pengujian beban (stress test) untuk mengukur throughput riil, batas konkurensi, dan jumlah maksimal shard yang efisien dikelola oleh satu router mongos.



## DAFTAR REFERENSI

- Brewer, E. (2012). CAP Twelve Years Later: How the “Rules” Have Changed. *IEEE Computer*, 45(2), 23–29.
- Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of Database Systems* (7th ed.). Pearson.
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O’Reilly Media.
- Koneru, N. M. K. (2025). Containerization Best Practices — Using Docker and Kubernetes for Enterprise Applications. *Journal of Information Systems Engineering and Management*, 10(45s), 724–746.
- MongoDB Inc. (2024). *MongoDB Manual: Sharding*. <https://www.mongodb.com/docs/manual/sharding/>
- MongoDB Inc. (2024). *MongoDB Manual: Replication*. <https://www.mongodb.com/docs/manual/replication/>
- Özsu, M. T., & Valduriez, P. (2020). *Principles of Distributed Database Systems* (4th ed.). Springer.
- vom Brocke, J., Hevner, A., & Maedche, A. (2021). *Introduction to Design Science Research*. Springer.